LECTURE : Quantum Mechanics for Computation

Code for reference :

REFERENCE: IBM learning Courses :

Quantum Information & Computation

Course - 1

"Basics of Quantum Information" by John Watrous

https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information

Bra and ket vector representation

In [1]:
```
from IPython.display import display, Math

# Print single states
display(Math(r'|0\rangle'))
display(Math(r'|1\rangle'))

display(Math(r'\langle0|'))
display(Math(r'\langle1|'))

# Print a superposition state
display(Math(r'\psi = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)'))
```

$$|0\rangle$$

$$|1\rangle$$

$$\langle 0|$$

$\langle 1|$

$\psi = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$

In [2]:
```
# Display Ket 0 as a column vector
display(Math(r'|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}'))

# Display Ket 1 as a column vector
display(Math(r'|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}'))
```

$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

In [3]:
```
# Display Bra 0 as a row vector
display(Math(r'\langle0| = \begin{pmatrix} 1 & 0 \end{pmatrix}'))

# Display Bra 1 as a row vector
display(Math(r'\langle1| = \begin{pmatrix} 0 & 1 \end{pmatrix}'))
```

$\langle 0| = \begin{pmatrix} 1 & 0 \end{pmatrix}$

$\langle 1| = \begin{pmatrix} 0 & 1 \end{pmatrix}$

In [4]:
```
import numpy as np
```

In [5]:
```
probs1=np.array([0.4,0.6])
probs2=np.array([0.3,0.7])
tensored_probs=np.kron(probs1,probs2)
print(tensored_probs)
```

```
[0.12 0.28 0.18 0.42]
```

In [6]:
```
print(np.sum(tensored_probs))
```

```
1.0
```

For the above classical probability states , the tensor product equals 1

Note: sum of all probabilities in the combined space equals 1.

Now, How to create quantum states?
Note: take the square root of the vector

In [7]:
```
vec1=np.sqrt(probs1)
vec1
```

Out[7]:
```
array([0.63245553, 0.77459667])
```

In [8]:
```
np.sum(vec1)
```

Out[8]:
```
np.float64(1.4070522012751594)
```

But now, the sum is not 1 ??

Q. How do we check valid quantum state?
ANS: norm, Euclidean norm to be used

In [9]:
```
np.linalg.norm(vec1)
```

Out[9]:
```
np.float64(1.0)
```

In [10]:
```
# Lets take the second state
vec2=np.sqrt(probs2)
print(vec2)
np.linalg.norm(vec2)
```

```
[0.54772256 0.83666003]
```

Out[10]:
```
np.float64(1.0)
```

Lets take tensored product of the 2 vectors

In [11]:
```
vec1=np.sqrt(probs1)
vec2=np.sqrt(probs2)
tensored_vec=np.kron(vec1,vec2)
print(tensored_vec)
```

```
[0.34641016 0.52915026 0.42426407 0.64807407]
```

Lets take the norm of the tensored vector which should be 1

In [12]:
```
np.linalg.norm(tensored_vec)
```

Out[12]:
```
np.float64(1.0)
```

"tensor product preserves the property of 1"

REFERENCE: qiskit.quantum_info

Quantum Information : Class to represent Quantum States

https://quantum.cloud.ibm.com/docs/en/api/qiskit/quantum_info

In [13]:
```
from qiskit.quantum_info import *
```

In [14]:
```
# we use a list of probabilities
Statevector(probs1)
```

```
Statevector([0.4+0.j, 0.6+0.j],
            dims=(2,))
```

To check the quantum state is valid or not ?

In [15]:
```
state_vec=Statevector(probs1)
state_vec.is_valid()
```

Out[15]: False

Reason: It is false , since its a probability vector

In [16]:
```
state_vec=Statevector(vec1)
state_vec.is_valid()
```

Out[16]: True

Reason: It is True, since we took the norm of the proabbility vector

Lesson learned: Intialization of a state vector

In [17]:
```
state_vec=Statevector(vec1)
state_vec
```

```
Statevector([0.63245553+0.j, 0.77459667+0.j],
            dims=(2,))
```

State vectors are array of numbers

Lets look at the example below:

In [18]:
```
vec=np.ones(3)
print("vec",vec)
vec /= np.linalg.norm(vec)
state_vec=Statevector(vec)
state_vec
```

```
vec [1. 1. 1.]
Statevector([0.57735027+0.j, 0.57735027+0.j, 0.57735027+0.j],
            dims=(3,))
```

A state vector above has equal values for all possible states.

Number of states: 3

So dim=3

what if it is 4,5,6,7,8...

In [19]:
```
vec=np.ones(4)
print("vec",vec)
vec /= np.linalg.norm(vec)
state_vec=Statevector(vec)
state_vec
```

```
vec [1. 1. 1. 1.]
Statevector([0.5+0.j, 0.5+0.j, 0.5+0.j, 0.5+0.j],
            dims=(2, 2))
```

In [20]:
```
# You ca change the value of the dim variable to 5,6,7,8..
dim=8
vec=np.ones(dim)
print("vec",vec)
vec /= np.linalg.norm(vec)
state_vec=Statevector(vec)
state_vec
```

```
vec [1. 1. 1. 1. 1. 1. 1. 1.]
Statevector([0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j,
             0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j,
             0.35355339+0.j, 0.35355339+0.j],
            dims=(2, 2, 2))
```

What we learn?

A state vector is initialized with a length, with a vector whose length is a power of 2,

It assumes that " vector is a state vector for qubits"

It means : 3 bits also we can say we have 3 qubits

Q. Can we have statevector with negative numbers ?

In [21]:
```
# A statevector with 3 qubits with negative numbers
vec= -np.ones(8)
print("vec",vec)
vec /= np.linalg.norm(vec)
```

```
state_vec=Statevector(vec)
state_vec
```

```
vec [-1. -1. -1. -1. -1. -1. -1. -1.]
Statevector([-0.35355339+0.j, -0.35355339+0.j, -0.35355339+0.j,
             -0.35355339+0.j, -0.35355339+0.j, -0.35355339+0.j,
             -0.35355339+0.j, -0.35355339+0.j],
            dims=(2, 2, 2))
```

In [22]:
```
state_vec.is_valid()
```

Out[22]: True

Yes we can have a statvector with negative numbers too .

We can too have complex numbers in statevectors

In [23]:
```
vec= -1j*np.ones(8)
print("vec",vec)
vec /= np.linalg.norm(vec)
state_vec=Statevector(vec)
state_vec
```

```
vec [0.-1.j 0.-1.j 0.-1.j 0.-1.j 0.-1.j 0.-1.j 0.-1.j 0.-1.j]
Statevector([0.-0.35355339j, 0.-0.35355339j, 0.-0.35355339j,
             0.-0.35355339j, 0.-0.35355339j, 0.-0.35355339j,
             0.-0.35355339j, 0.-0.35355339j],
            dims=(2, 2, 2))
```

In [24]:
```
state_vec.is_valid()
```

Out[24]: True

Q. How to do measurement with a state vector ?

In [25]:
```
# We sample 10 counts
state_vec.sample_counts(10)
```

Out[25]:
```
{np.str_('000'): np.int64(2),
 np.str_('010'): np.int64(1),
 np.str_('011'): np.int64(1),
 np.str_('101'): np.int64(2),
 np.str_('110'): np.int64(3),
 np.str_('111'): np.int64(1)}
```

Have a look at the bit strings reperesentation

--> length = 8

--> no of bits = 3 i.e 2^3= 8

sample_counts: dictionary of measurement outcomes and the number of times each specific state was observed during a quantum circuit execution.

https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector

We can also say "sample_counts simulates the measurement of a quantum state"

In the next example it Simulates 1000 measurements (shots) on the state (state_vec)

In [26]:
```
# Lets consider 1000 counts
state_vec.sample_counts(1000)
```

Out[26]:
```
{np.str_('000'): np.int64(126),
 np.str_('001'): np.int64(121),
 np.str_('010'): np.int64(135),
 np.str_('011'): np.int64(134),
 np.str_('100'): np.int64(130),
 np.str_('101'): np.int64(110),
 np.str_('110'): np.int64(124),
 np.str_('111'): np.int64(120)}
```

output is in the form of Dictionary

keys --> measured binary bitstrings (e.g., '000', '111' so on)

values --> corresponding number of times those bitstrings were observed

Lets take probabilties

In [27]:
```
state_vec.probabilities()
```

Out[27]:
```
array([0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125])
```

It shows absolute value of squared. For a complex number we take the value squared and we get a positive real number.

In [28]:
```
vec= -1j*np.ones(8)

#print("vec",vec)
vec /= np.linalg.norm(vec)
state_vec=Statevector(vec)
print("\nprobabiltities dict:")
state_vec.probabilities_dict()
```

```
probabiltities dict:
```

Out[28]:
```
{np.str_('000'): np.float64(0.12499999999999997),
 np.str_('001'): np.float64(0.12499999999999997),
 np.str_('010'): np.float64(0.12499999999999997),
 np.str_('011'): np.float64(0.12499999999999997),
 np.str_('100'): np.float64(0.12499999999999997),
 np.str_('101'): np.float64(0.12499999999999997),
 np.str_('110'): np.float64(0.12499999999999997),
 np.str_('111'): np.float64(0.12499999999999997)}
```

It shows absolute value squared.

Lesson outcome : learned about sampling conecpt using sample_counts(), probabilities_dict().

Now lets consider a single qubit state vector

Using from_label method

In [29]:
```
statevec=Statevector.from_label("0")
statevec
```

```
Statevector([1.+0.j, 0.+0.j],
            dims=(2,))
```

It intializes a single qubit in zero state

(we have 1 as the first entry) as seen above

In [30]:
```
statevec=Statevector.from_label("1")
statevec
```

```
Statevector([0.+0.j, 1.+0.j],
            dims=(2,))
```

Now we have initialized a single qubit in one state

(we have 1 as the second entry)

Lets sample_counts for 1000 shots and observe the output ?

In [31]:
```
statevec.sample_counts(1000)
```

Out[31]:
```
{np.str_('1'): np.int64(1000)}
```

We always get 1

Similarly if you check for zero satate

In [32]:
```
statevec=Statevector.from_label("0")
statevec.sample_counts(1000)
```

Out[32]:
```
{np.str_('0'): np.int64(1000)}
```

We always get 0

Now we perform some operations on the statevector

In [33]:
```
from qiskit.circuit.library import XGate
```

In [34]:
```
statevec=Statevector.from_label("0")
statevec = statevec.evolve(XGate())
statevec.sample_counts(1000)
```

Out[34]:
```
{np.str_('1'): np.int64(1000)}
```

X gate flips the qubit from zero state to one state

Lets use RXGate ()

In [35]:
```
from qiskit.circuit.library import RXGate, RYGate, RZGate
x= [0.1,0.2,0.3,0.4,0.7,1.0,1.5,2.0,2.6,3.0]

for i in range(len(x)):

    statevec=Statevector.from_label("0")
    statevec = statevec.evolve(RXGate(x[i]))
    print(x[i] , " ",statevec.sample_counts(1000))
```

```
0.1   {np.str_('0'): np.int64(997), np.str_('1'): np.int64(3)}
0.2   {np.str_('0'): np.int64(993), np.str_('1'): np.int64(7)}
0.3   {np.str_('0'): np.int64(979), np.str_('1'): np.int64(21)}
0.4   {np.str_('0'): np.int64(966), np.str_('1'): np.int64(34)}
0.7   {np.str_('0'): np.int64(903), np.str_('1'): np.int64(97)}
1.0   {np.str_('0'): np.int64(743), np.str_('1'): np.int64(257)}
1.5   {np.str_('0'): np.int64(533), np.str_('1'): np.int64(467)}
2.0   {np.str_('0'): np.int64(259), np.str_('1'): np.int64(741)}
2.6   {np.str_('0'): np.int64(62), np.str_('1'): np.int64(938)}
3.0   {np.str_('0'): np.int64(4), np.str_('1'): np.int64(996)}
```

Observation : As we increase the argument value from 0.1 to 3 we have increased the probability chances of obtaining 1

Unitary matrix representation for RXGate

In [36]:
```
gate = RXGate(0.1)
rxGate_matrix= np.array(gate)
rxGate_matrix
```

Out[36]:
```
array([[0.99875026+0.j        , 0.        -0.04997917j],
       [0.        -0.04997917j, 0.99875026+0.j        ]])
```

In [37]:
```
from qiskit.visualization import array_to_latex
array_to_latex(rxGate_matrix)
# using array to latex to represent  the unitary matrix
```

Out[37]:

$$\begin{bmatrix} 0.9987502604 & -0.0499791693i \\ -0.0499791693i & 0.9987502604 \end{bmatrix}$$

In [38]:
```
gate = RXGate(0.5)
rxGate_matrix= np.array(gate)
array_to_latex(rxGate_matrix)
```

Out[38]:

$$\begin{bmatrix} 0.9689124217 & -0.2474039593i \\ -0.2474039593i & 0.9689124217 \end{bmatrix}$$

To check if the above matrix is unitary we need to multiply with its conjugate

In [39]:
```
rxGate_matrix @ rxGate_matrix.T.conj()
```

Out[39]:
```
array([[1.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j]])
```

We get a Identity matrix.

Conclude that the matrix of this gate will always will be unitary, which is true for quantum states

In [40]:
```
rxGate_matrix.T.conj() @ rxGate_matrix
```

Out[40]:
```
array([[1.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j]])
```

It doesn't matter which side we perform conjugate , we still get an Indentity matrix

In [41]:
```
from qiskit.visualization import array_to_latex
array_to_latex(rxGate_matrix.T.conj() @ rxGate_matrix)
```

Out[41]:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

How to visualize a single qubit state

Called "psi" , which is a linear combination of zero state and one state as shown:

$$\ket{\psi} = \alpha\ket{0} + \beta\ket{1}$$

$$|\alpha|^2 + |\beta|^2 = 1$$

$$\alpha, \beta$$

are complex co-effcients

The absolute value squared to 1. It is requirement fro a valid state.

Using trignometric functions, we can write psi as :

$$\ket{\psi} = \cos\frac{\theta}{2}\ket{0} + \sin\frac{\theta}{2}\ket{1}$$

$$\cos^2(\theta/2) + \sin^2(\theta/2) = 1$$

This is also a valid quantum state

These two are complex numbers.

Lets introduce a phase on one state

$$\ket{\psi} = \cos\frac{\theta}{2}\ket{0} + e^{i\varphi}\sin\frac{\theta}{2}\ket{1}$$

Now we have a relative phase with these 2 co-effcients

We can also have an another phase

This general expression for a state is written as:

$$\ket{\psi} = e^{i\gamma}\left(\cos\frac{\theta}{2}\ket{0} + e^{i\varphi}\sin\frac{\theta}{2}\ket{1}\right)$$

$$\gamma, \varphi$$

are interpreted as angles

In [42]:
```python
import cmath, math

def bloch_angles(a: complex, b: complex) -> tuple[float, float]:
    global_phase = cmath.rect(1, cmath.phase(a))
    a *= global_phase.conjugate()
    b *= global_phase.conjugate()
```

```
    assert math.isclose(a.imag, 0, abs_tol=1e-12)
    return 2 * math.acos(a.real), cmath.phase(b)
```

In [43]:
```
statevec = Statevector.from_label("0")

print(statevec)

bloch_angles(*statevec)
```

```
Statevector([1.+0.j, 0.+0.j],
            dims=(2,))
```

Out[43]: (0.0, 0.0)

In [44]:
```
statevec = Statevector.from_label("0")
statevec = statevec.evolve(RXGate(3))

print(bloch_angles(*statevec))

statevec.sample_counts(1000)
```

```
(3.0, -1.5707963267948966)
```

Out[44]: {np.str_('0'): np.int64(4), np.str_('1'): np.int64(996)}

In [45]:
```
statevec = Statevector.from_label("0")
statevec = statevec.evolve(RXGate(0.1))

print(bloch_angles(*statevec))

statevec.sample_counts(1000)
```

```
(0.09999999999999855, -1.5707963267948966)
```

Out[45]: {np.str_('0'): np.int64(1000)}

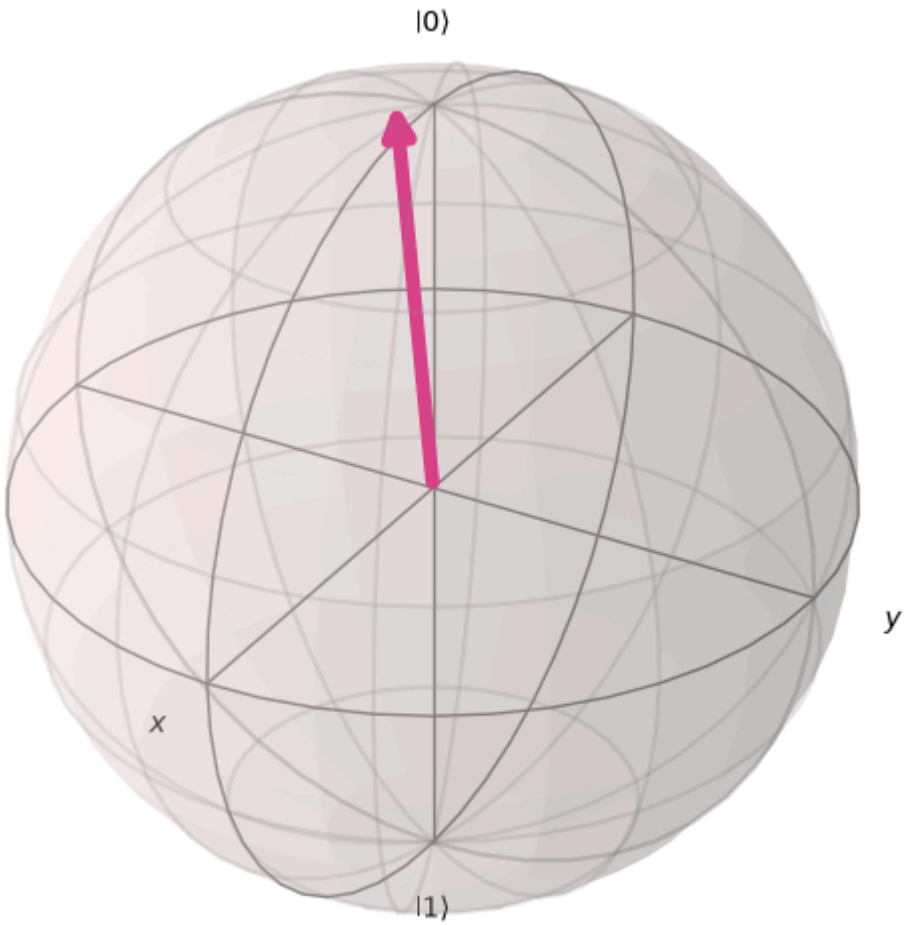
Another way to visualize using plot_bloch_vector from visualization

In [46]:
```
from qiskit.visualization import plot_bloch_vector

theta, phi = bloch_angles(*statevec)
```

```
plot_bloch_vector([1, theta, phi], coord_type="spherical")
```

Out[46]:



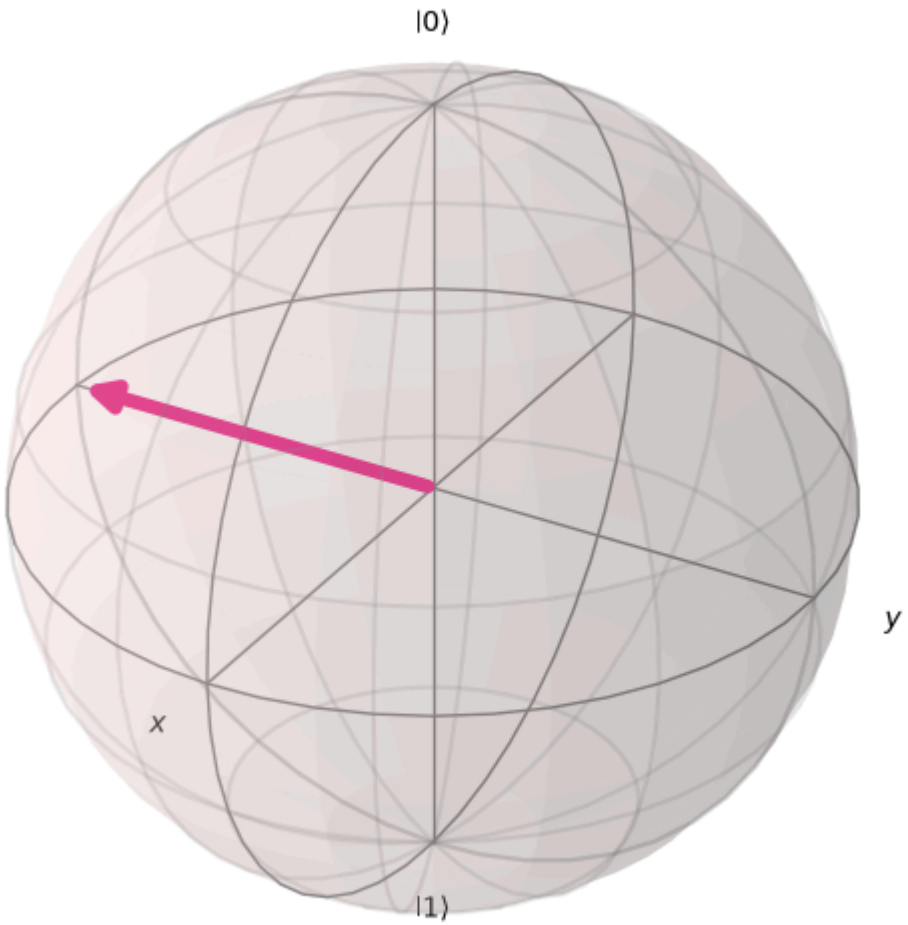
Lets change the RX angle to pi/2

In [47]:

```
statevec = Statevector.from_label("0")
statevec = statevec.evolve(RXGate(np.pi/2))
```

```
theta, phi = bloch_angles(*statevec)
plot_bloch_vector([1, theta, phi], coord_type="spherical")
```

Out[47]:



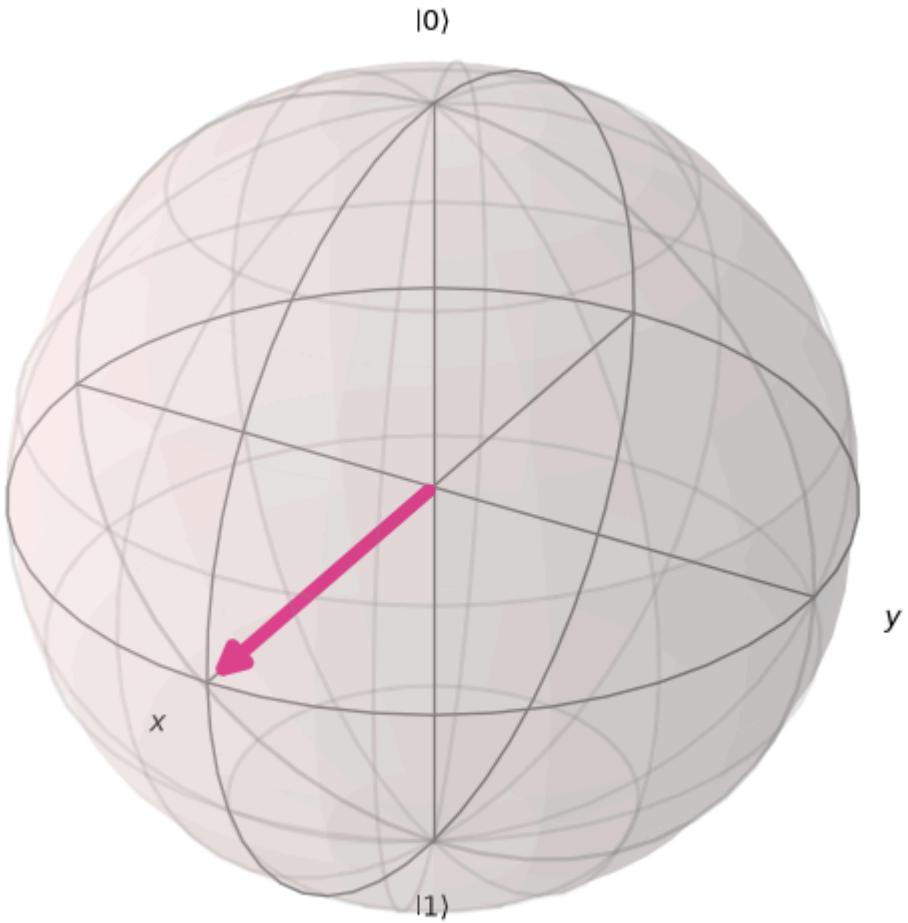
Lets add the RZ angle with pi/2

In [48]:

```
statevec = Statevector.from_label("0")
statevec = statevec.evolve(RXGate(np.pi/2))
statevec = statevec.evolve(RZGate(np.pi/2))
```

```
theta, phi = bloch_angles(*statevec)
plot_bloch_vector([1, theta, phi], coord_type="spherical")
```

Out[48]:



Lets add RY gate with angle pi/2

In [49]:
```
statevec = Statevector.from_label("0")
statevec = statevec.evolve(RXGate(np.pi/2))
statevec = statevec.evolve(RZGate(np.pi/2))
statevec = statevec.evolve(RYGate(np.pi/2))

theta, phi = bloch_angles(*statevec)
plot_bloch_vector([1, theta, phi], coord_type="spherical")
```

Out[49]: